

Tip: There are very few tools in this space. License Finder from Pivotal works very well and is open source as well.

How about application runtimes (containers)?

Docker containers have been used widely to ship the application with its runtime and its operating system dependencies. Each container image consists of a set of open source packages like kernel, glibc, shell, utils, etc. These system packages may contain security issues as well. So scanning the container images is inevitable. Choosing the right container image is crucial. The criteria for choosing a container image are:

- It's officially published by the Docker community or the library maintainer (check whether your publisher has any of the image tags given in Figure 11).
- It has a small footprint (like alpine).
- It is updated regularly.
- A new image is published in case of new vulnerabilities.

We can use a distroless image from Google as an option.

The next step involves scanning the container images for known vulnerabilities. There are a lot of open source tools available to scan container images. My preference is Anchore open source. Again, choose a tool that uses all the databases. For example, Trivy includes all public databases.

Common mistakes

You can avoid common mistakes while building container images, as indicated below.

- Use multi-stage builds and reduce the image footprint. For example, to run any *go* applications, we can use the *go lang* image to build the artifact and copy it to an alpine image.

```
FROM golang:1.7.3
WORKDIR /go/src/github.com/alexellis/href-counter/
```



Figure 10: Tools | OSS licence compliance checkers

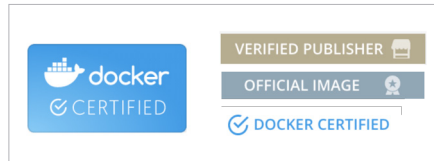


Figure 11: Docker hub official image tag formats

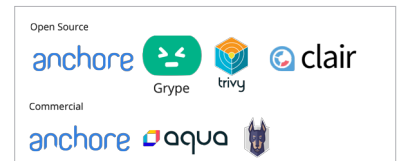


Figure 12: Tools | Container scanning tools

Public Database			
nvd	centos	alpine	python-safety-db
redhat	fedora	photon	ghsa-composer
redhat-oval	amazon	ruby-advisory-db	ghsa-maven
debian	oracle-oval	rust-advisory-db	ghsa-npm
debian-oval	suse-cvrf	php-security-advisories	ghsa-nuget
ubuntu	opensuse-cvrf	nodejs-security-wg	ghsa-pip
			ghsa-rubygems
Private Paid Database			
 VulnDB			

Figure 13: Vulnerability data sources

```
RUN go get -d -v golang.org/x/net/html
COPY app.go .
RUN CGO_ENABLED=0 GOOS=linux go build
-a -installsuffix cgo -o app .
FROM alpine:latest
RUN apk --no-cache add ca-certificates
WORKDIR /root/
COPY --from=0 /go/src/github.com/alexellis/href-counter/app .
CMD ["/app"]
```

- Install additional packages without storing the package manager's cache information:

```
RUN apk --no-cache add ca-certificates
```

- Run the application as non-root user.

Use automated tools to check instructions in the *dockerfile*. Dockle supports auditing CIS benchmarks as well.

Hardening infrastructure scripts

Security misconfiguration is at the top in the NSA list of cloud vulnerabilities. Modern applications are more than just the code developers create. Since the infrastructure is written as code, we can perform automated analysis of the Infrastructure as a Code (IaC) templates to spot potential security issues, or misconfiguration and compliance violations. This way you can mitigate risks before provisioning cloud native infrastructure.



Figure 14: Tool | Infrastructure as code hardening checker